

ORIE 5355/INFO 5370 HW 3: Algorithmic Pricing

- Name:
- Net-id:
- Date:
- Late days used for this assignment:
- Total late days used (counting this assignment):
- People with whom you discussed this assignment:

Notes

We have marked questions in **green**. Please put answers in the default color. You'll want to write text answers in "markdown" mode instead of code. In Jupyter notebook, you can go to Cell > Cell Type > Markdown, from the menu. Please carefully read the late days policy and grading procedure [here](#). In that link, we also give some tips on exporting your notebook to PDF, which is required for GradeScope submission.

Some of the questions can be interpreted in multiple ways. That is always true in data science. You'll need to make judgment calls for what analysis to do. For the homework, you'll still receive full points for any "reasonable" choice. Briefly explain and justify your choices. Also feel free to ask questions on EdStem.

Disclosure and survey

Type "I understand" in the next box to attest to the following statements.

I understand that I have to submit this homework assignment in two places:

- gradescope -- where I will submit a pdf of the assignment and tag each page of my assignment with the problem to which it corresponds.
- github (via classroom50) -- where I will commit and push up this notebook and the rest of my code, in a runnable format.

I understand that I will reach out to the teaching staff via office hours or EdStem at least 5 days before the homework is due if I have any issues submitting the assignment on either of these platforms, such as I am not on the class roster. I understand that I may receive points off (up to full value of the homework assignment) if I do not submit to both locations.

I understand that the teaching staff prefers that I make multiple commits on github (via

classroom50) as I am completing the assignment (for example, after I finish each problem), and that this information may be used to assess overall understanding, progress, and qualitative assessment/grading.

I understand that I must fully disclose my usage of AI in the box below **and** at the top of each problem. I will follow the AI usage policy, and am responsible for everything that I submit. I understand that quizzes and other assessments may evaluate understanding of what I submitted, including asking me to interpret what code does.

After you finish the homework, please complete the following (short, anonymous) post-homework [survey](#) and include the survey completion code below.

AI disclosure. In the below box, please write 1-3 sentences about your use of AI in the assignment **overall**. Example responses include:

- None
- "I vibe-coded all the code for the assignment. I didn't even try to understand or parse the code"
- "I referred to ChatGPT for interpreting parts of the assignment, and to help me write code, but ultimately I understood each of its outputs and either typed/copied in small pieces code that it gave me. I wrote more than half the code myself".

Note that AI disclosure is part of the grade for each component, and you are ultimately responsible for your learning.

Please also note that AI disclosure is part of each question below

Conceptual component [4 points]

AI disclosure for this conceptual component: In the next box, describe any AI use for this conceptual component and how it was used, or write "None".

Please complete the following pricing ethics scenario questionnaire: <https://forms.gle/Bahbqcn9126u3DzM7>, and include the survey completion code below. **We will discuss these issues in class on 10/19. You must complete the questionnaire before the day of that class, even if you turn in the rest of the homework later. The questionnaire will close the morning of the class that we discuss these issues.**

Survey completion code:

Based on the first letter of your first name, explain your answers to the following questions, in at most three sentences each.

First letter A-C: 1, 6, 11, 16

First letter D-H: 2, 7, 12, 17

First letter I-M: 3, 8, 13, 18

First letter N-S: 4, 9, 14, 19

First letter T-Z: 5, 10, 15, 20

Give and discuss your answers for the 4 questions in this notebook, to be turned in with the rest of the homework. Also be prepared to discuss your answers to at least these questions in class (I might randomly call on people), but you should also be willing/able to discuss your answers to other questions.

Programming component

Helper code

```
In [16]: import numpy as np
import pandas as pd
import os, sys, math
import matplotlib.pyplot as plt
```

```
In [17]: df_train = pd.read_csv('HW3_data_train.csv')
test_demand_curve = pd.read_csv('test_demand.csv')
```

```
In [18]: df_train.head()
```

```
Out[18]:
```

	Location	Income	Offered price	Purchased
0	Africa	10.38	3.16	False
1	Europe	26.33	3.47	True
2	Europe	24.06	3.78	True
3	Africa	16.18	3.74	False
4	Asia Pacific	13.73	4.75	False

```
In [19]: df_train.shape, test_demand_curve.shape
```

```
Out[19]: ((4000, 4), (199, 5))
```

Problem 1: Demand estimation and pricing without covariates

AI disclosure for this problem: In the next box, describe any AI use for this problem and how it was used, or write "None".

First, we will use the training data to construct estimates of the demand at each price without leveraging the covariates, and then use that estimated function to calculate optimal prices.

```
In [ ]:
```

1a) Naive method: empirical estimate of demand $d(p)$ at each price

Fill in the below function, that takes in a dataframe and the number of bins into which to separate the historical prices. The function should output a dataframe that has one row for each price bin, with two columns: the bin interval, and the estimated demand $d(p)$ (the fraction of potential customers who purchase at price p) in that bin.

Use the following function to create bins: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.qcut.html>

```
In [20]: # Example with 10 bins:
df_train['bin_with_10_bins'] = pd.qcut(df_train['Offered price'], 10)
df_train.head()
```

```
Out[20]:
```

	Location	Income	Offered price	Purchased	bin_with_10_bins
0	Africa	10.38	3.16	False	(2.95, 3.54]
1	Europe	26.33	3.47	True	(2.95, 3.54]
2	Europe	24.06	3.78	True	(3.54, 4.19]
3	Africa	16.18	3.74	False	(3.54, 4.19]
4	Asia Pacific	13.73	4.75	False	(4.19, 4.77]

For example, with 2 bins and passing in `df_train` to the function, you should see the following output:

	Price_bin	Demand_at_price
0	(0.499, 3.54]	0.583417
1	(3.54, 6.5]	0.050050

(If you can't see the image, then it is an image with 2 columns (Price_bin and Demand_at_price), and two rows. The First row has price bin (0.499, 3.54] and Demand_at_price equal to .583.)

```
In [21]: def create_empirical_estimate_demand_df(df, number_of_pricing_bins):
pass
```

In []:

Fill in the below function, that takes in a single price and your empirical df from the above function and outputs the prediction for the demand $d(p)$ at that price. For example, with 2 bins, at price = 3 the function should output 0.583417.

If the price is lower than the smallest bin, then use the value of the smallest bin. If it is higher than the highest bin, use the value of the highest bin.

```
In [22]: def get_prediction_empirical(empirical_df, price):
pass
```

In []:

```
In [23]: prices_to_predict = np.linspace(min(df_train['Offered price']), max(df_train
```

Plot in a single figure the outputs of your function as a line plot -- where the X axis corresponds to prices in `prices_to_predict` and the Y axis the predicted Demand at that price -- for the following three inputs to the function:

1. the dataframe is the first 100 rows of `df_train`, with 10 bins.
2. the dataframe is the first 500 rows of `df_train`, with 10 bins.
3. the dataframe is all the rows of `df_train`, with 10 bins.

In the same figure, include the "true" test-time demand curve, using `test_demand_curve` that we loaded at the top of the homework -- plot the mid-point of each bin on the X axis, and the demand for that bin on the Y axis. So your plot will contain 4 curves in total.

In []:

In []:

Do the same plot, except now you're using 50 bins for each of the three data frames.

In []:

Comment on your output in no more than 3 sentences. What is the effect of using more data and more bins?

1b) Demand estimation using logistic regression

First, Fill in the below function that fits a logistic regression to predict the probability of purchase at a price ($d(p)$). The logistic regression should just have two coefficients: one for the intercept, and one for the price. The function takes in a dataframe that you will use as your training data for your model, and should return your fitted model.

Note: the "intercept" term corresponds to a column of all 1s in your dataframe. If you do logistic regression in statsmodels, you can have it add the intercept term for you if you use the formula feature. In sklearn, there is a parameter called `fit_intercept`.

```
In [24]: def fit_logistic_regression_demand_just_on_price(df):  
         pass
```

Fill in the below function, that takes in a single price and your trained model and outputs the prediction for the demand $d(p)$ at that price.

Return the predicted probability of `Purchased=True` (equivalently, 1), not a binary class prediction. With scikit-learn, use `fitted_model.predict_proba(X)` and select the column for the `True / 1` class, checking `fitted_model.classes_`. With a fitted statsmodels logistic model, use `fitted_model.predict(X)` to obtain purchase probabilities. In both cases, construct the prediction inputs consistently with the model you fitted, including the intercept where required.

```
In [25]: def get_prediction_logistic(fitted_model, price):  
         pass
```

For each of the three training dataframes as in part A, fit a model and get the predictions for each of the prices in `prices_to_predict` using your above function. Generate the same lineplot as above. Also include the "true" test-time demand curve, `test_demand_curve`.

In []:

Comment on your output in no more than 3 sentences. What is the effect of using logistic regression instead of the empirical distribution?

1c) Optimal pricing using your demand estimates

For this part, use 10 bins for each empirical model and use `prices_to_predict` as the candidate-price grid for every model and the supplied test curve. Here, "optimal" means the best price on this grid, not the best price over all real numbers. If multiple prices maximize revenue, choose the smallest. Use the same grid and test-curve lookup convention in Problem 2b.

To evaluate a supplied test curve at price p , use `Demand_at_price` from the row with `Price_bin_left < p <= Price_bin_right`. At or below the smallest left endpoint, use the first row; above the largest right endpoint, use the last row. Treat demand as constant within each bin; do not interpolate between bin midpoints.

Fill in the following function that takes in two lists: a list of prices, and a list of predicted demand $d(p)$ at that price. The function outputs the revenue maximizing price given the data and the corresponding revenue. You may use a "brute force" technique, that loops through all the possible prices and calculates the revenue using that price.

```
In [26]: def get_revenue_maximizing_price_and_revenue(price_options, demand_predictions):  
pass
```

```
In [ ]:
```

Print out the optimal price and the predicted optimal revenue from the predictions for your naive and logistic models, using 100 rows and all the data, each.

For example, we got the following:

logistic, 100 points: 2.580402010050251 1.2143341610705582

naive, all points: 2.278894472361809 1.348157868550674

Note: last year, people reported different numbers for the logistic regression based on the exact package they used (which might, for example, have different default solvers or default parameters. If you get a different number, that's fine, as long as your methods are correct.) For example, with scikit-learn, people last year reported getting: (2.1884422110552766, 1.4070878495510661) for the logistic regression with 100 rows.

```
In [ ]:
```

Now, we're going to use the "true" test-time demand curve, `test_demand_curve`. For each of the above predicted optimal prices, calculate the revenue resulting from that price used on the true demand curve. Also print out the true optimal price and corresponding revenue for that curve on `prices_to_predict`, using the bin-lookup convention above.

For example, we got:

true revenue using logistic 100 price: 0.9729384628058323

(Again, you might get a slightly different number, depending on how you implemented your code. That's OK.)

In [1]:

How do your estimates compare to the actual revenue? Discuss in no more than 3 sentences.

Problem 2: Demand estimation and pricing with covariates

AI disclosure for this problem: In the next box, describe any AI use for this problem and how it was used, or write "None".

Now, we are going to ask you to do personalized pricing, based on just two binary covariates.

First, compute the median `Income` using all of `df_train`. Create a wealth indicator: `0` (`LowWealth`) if `Income` is at or below this median, and `1` (`HighWealth`) if `Income` is strictly above it. Keep this cutoff fixed when making predictions. Second, create a location indicator: `1` (`America`) if the location is America, and `0` (`NotAmerica`) otherwise.

For this section, we will use all the `df_train` data, as opposed to just the first few rows. Write x for the pair of binary covariates, and $d(p,x)$ for the probability of purchase at price p for a customer with covariates x .

2a) Demand estimation

First, fill in the below function that fits a logistic regression to predict the probability of purchase $d(p,x)$. Retain the intercept and offered-price predictor from Problem 1, and add one coefficient for each of the two binary covariates. You may also add interactions, including interactions between price and covariates. If you add more interactions, you may wish to add regularization.

In [27]: `def fit_logistic_regression_demand_with_covariates(df):
 pass`

Fill in the below function, that takes in a single price, covariates, and your trained model, and outputs the prediction for the demand $d(p,x)$ at that price. For example, one covariate combination is ['NotAmerica', 'LowWealth']; encode its indicators consistently with your fitted model.

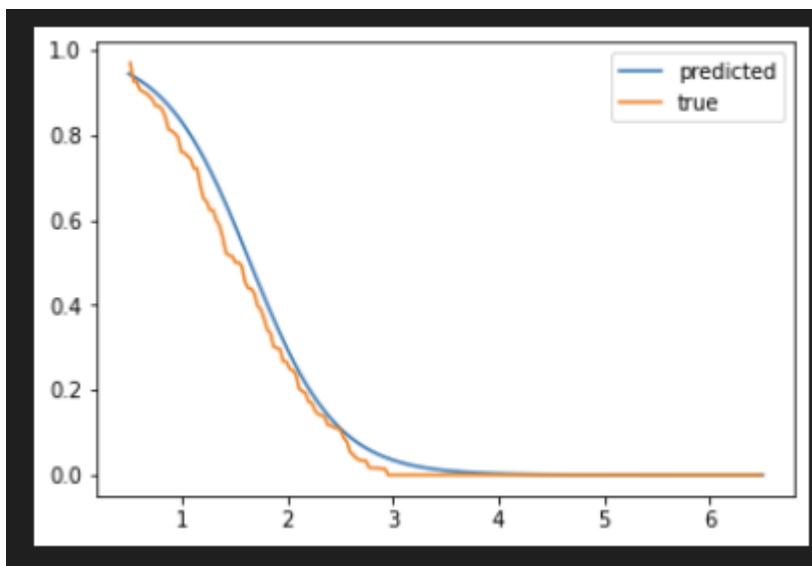
```
In [28]: def get_prediction_logistic_with_covariates(fitted_model, price, covariates):
         pass
```

```
In [29]: test_demand_curve_America_HighWealth = pd.read_csv('test_demand_America_HighWealth.csv')
         test_demand_curve_NotAmerica_HighWealth = pd.read_csv('test_demand_NotAmerica_HighWealth.csv')
         test_demand_curve_America_LowWealth = pd.read_csv('test_demand_America_LowWealth.csv')
         test_demand_curve_NotAmerica_LowWealth = pd.read_csv('test_demand_NotAmerica_LowWealth.csv')
```

Fit a model on the full training dataframe, then get the predictions for each of the prices in `prices_to_predict` using `get_prediction_logistic_with_covariates` and each unique covariate combination.

For each covariate combination, generate the same lineplot as in 1a and 1b (separately for each covariate combination). Also include the "true" test-time demand curve for the appropriate covariate combination

For example, for `NotAmerica_LowWealth`, we got (again, your answers may differ slightly):



```
In [ ]:
```

2b) Pricing

Now, use your code from 1c to output predicted optimal prices, predicted revenue, and the actual revenue using the test data curve, for each covariate combination. Your output should have 4 rows, one for each covariate combination.

For example, we got for one of the covariate combinations:

Group Not America, Low Wealth: Optimal price 1.314070, Predicted Revenue 0.913284, True revenue 0.788442

In []:

Suppose each of the 4 covariate combinations make up an equal part of the population. What would be the resulting revenue achieved at test time if you use the optimal price for each group (so you look at their covariates, and then give them the optimal price for that group).

In other words, this answer should be a single number, the mean of the true revenue of the 4 groups.

In []:

Comment on your outputs in no more than 3 sentences. What is the effect of using different prices for different covariate groups?

Problem 3: Pricing under capacity constraints

AI disclosure for this problem: In the next box, describe any AI use for this problem and how it was used, or write "None".

Now, we are going to build up to implementing the Bellman equation approach discussed in class, to price a single copy of an item to be sold over T time periods. One potential customer arrives in each period $t=0, \dots, T-1$. Conditional on the item still being available, the customer purchases with probability $d(p)$ when offered price p ; this probability does not change over time. After a sale, the item is no longer available. An item left unsold at time T has no salvage value. For simplicity, use `test_demand_curve` as $d(p)$ and its bin midpoints as the candidate prices for Problem 3, as specified in the next cell.

In [30]:

```
price_options = list(test_demand_curve.Price_bin_mid)
demand_predictions = list(test_demand_curve.Demand_at_price)
```

3a) Implementing one step of the Bellman equation

Recall the "Bellman equation" taught in class. Let V_t be the maximum expected remaining total revenue from periods t through $T-1$, conditional on still having the

item at the start of period t . We set $V_T=0$. If p_t is the optimal price in period t , then: $V_t = d(p_t)p_t + (1 - d(p_t))V_{t+1}$

Implement the following function that returns optimal price p_t and the resulting value V_t , given the demand curve and V_{t+1} .

For example, we find that the output of the following function call is: (2.083, 1.1922434210526316)

```
get_single_step_revenue_maximizing_price_and_revenue(0,
price_options, demand_predictions)
```

```
In [31]: def get_single_step_revenue_maximizing_price_and_revenue(Vtplus1, price_opti
pass
```

```
In [ ]:
```

3b) Calculating prices over time

Implement the following function that returns a list (of length T) of optimal prices for each time period, and an expected remaining total revenue V_0 for those prices, starting with the item unsold.

Hint: your function should loop through each time step, *starting* at time $t=T-1$ (the last time period, since the first time period is time $t=0$). Each iteration of the loop should call the function from part 3a. Recall that we can define $V_T = 0$, since even if the item is unsold at time T , we have finished trying to sell it.

```
In [32]: def get_prices_over_time_and_expected_revenue(prices, demand_predictions, T)
pass
```

Plot a line plot for your optimal prices over time when $T=100$ and $T=10$. Also print out the expected revenue using these prices and for each T .

For example, when $T=100$, we find that $prices[0] = 5.822$, $prices[90] = 4.224$, and that revenue = 5.2287

```
In [ ]:
```

3c) Prices over time with multiple copies

Now, suppose that you have K copies of the item to sell over T time periods. As above, one potential customer arrives per period and buys at most one item with stationary probability $d(p)$. Let $V_{t,k}$ be the maximum expected remaining total revenue from periods t through $T-1$ when k items remain at the start of period t . Use the boundaries $V_{T,k}=0$ for every k and $V_{t,0}=0$ for every t .

Implement the two-dimensional dynamic program as discussed in class.

For $T=100$ and $K=5$, plot the optimal price $p_{t,k}$ against time $t=0,\dots,T-1$, with one curve for each $k=1,\dots,K$. Each curve shows the price at time t when k items remain.

Hint: As in 3a and 3b, you may find it useful to first optimize the price $p_{t,k}$ given the values $V_{t+1,k}$ and $V_{t+1,k-1}$. Then, have a second function that loops through t,k in an appropriate order.

In []: